JoeLang 1.0

User Manual

A typed, object-facing programming language

Language version: 1.0
Document type: User reference

November 8, 2026

Contents

1 Introduction **2**
1.1 First program . . . 2
1.2 Top-level declarations . . . 2

2 Keywords and names **2**

3 Types, literals, and conversion **3**
3.1 Nullable references . . . 3

4 Expressions **3**
4.1 Text and logic . . . 4

5 Variables and scope **4**

6 Conditions and loops **4**
6.1 Conditions . . . 4
6.2 Counting loops . . . 4
6.3 Collection loops . . . 5

7 Fixed Joerays **5**

8 Joebjects and behaviors **5**

9 Input **6**

10 Packages **6**

11 Standard packages **6**
11.1 JoeMath: `std/joemath` . . . 6
11.2 JoerayList: `std/joeraylist` . . . 7
11.3 JoeTime: `std/joetime` . . . 7
11.4 JoeText: `std/joetext` . . . 7

12 Diagnostics **7**

13 Quick reference **7**

1 Introduction

JoeLang is a typed, object-facing programming language. A program may be a short top-level script, or it may define *Joebjects*: named types that own state and receive messages through behaviors. Brace-delimited blocks, explicit types, and zero-based indexing are intended to feel familiar to Java and C++ programmers.

JoeLang is case-sensitive. The spellings `joet`, `JOT`, and `jint` are distinct. A newline normally ends a statement; a semicolon may also end one. Indentation improves readability but does not change meaning.

1.1 First program

```
show "Hello, Joe!"
```

The `show` statement writes a value to the program output. A Joebject is not required for a short program.

1.2 Top-level declarations

```
has strjin greeting = "Welcome"
has jint launch_count = 0
show greeting
```

At top level, `has` creates a program variable.

2 Keywords and names

Names begin with a letter or underscore and may contain letters, digits, and underscores. A name may not be a reserved word.

Keywords	Purpose
`joebject`, `does`, `newjoe`, `self`	Object declaration, behavior declaration, construction, and current receiver
`has`, `give`, `as`	Declaration, behavior result, and alias syntax
`show`, `input`	Output and runtime input
`when`, `otherwise`, `joeach`, `jin`	Conditions and loops
`jinport`	Package import
`jint`, `joeble`, `joelean`, `jong`, `strjin`, `type`	Types and conversions
`joet`, `joef`, `gunmu`, `JAND`, `JOR`, `JOT`	Values and logical operators

The earlier spellings `each`, `in`, `ask`, `from`, `truejoe`, `falsejoe`, and `nonejoe` are not JoeLang 1.0 keywords.

3 Types, literals, and conversion

Every declaration and behavior parameter has a type.

Type	Meaning	Example
`jint`	signed 32-bit whole number	`42`
`joeble`	double-precision decimal number	`3.14`
`joelean`	logical value	`joet`
`jong`	signed 64-bit whole number	`42J`
`strjin`	immutable text	`"Joe"`

```
has jint score = 92
has joeble average = 91.5
has joelean passed = joet
has jong visitors = 9000000000J
has strjin name = "JoeJin"
```

Assignment requires the same type. JoeLang does not silently narrow or widen a value during assignment. Use `type` for an explicit conversion.

```
has jint whole = type.jint(2.9)
has joeble decimal = type.joeble(whole)
has jong long_total = type.jong(whole)
```

`type.jint` truncates toward zero. Invalid conversion, non-finite decimal conversion, and numeric overflow report a source-mapped runtime error.

3.1 Nullable references

`gunmu` represents intentional absence. It is valid only for a nullable reference type, written with `?`.

```
has strjin? nickname = gunmu
nickname = "Jin"
```

Numeric values and `joelean` values are not nullable.

4 Expressions

JoeLang uses familiar precedence: postfix operations, unary operations, multiplication and division, addition and subtraction, comparisons, equality, `JAND`, then `JOR`. Parentheses make order explicit.

```
has jint subtotal = 8 + 4 * 2
has jint quotient = 7 / 2
has joeble exact = 7 / 2.0
has jint remainder = 7 % 2
```

Integral division truncates toward zero. `jint` with `jint` produces `jint`; an expression containing `jong` produces `jong` unless it also contains `joeble`; any expression containing `joeble` produces `joeble`. Integral overflow is an error.

4.1 Text and logic

```
has jint score = 92
show "Score: " + score
has joelean eligible = score >= 60
has joelean active = joet JAND JOT joef
```

Adding a `strjin` to a scalar produces text. `JAND` and `JOR` short-circuit. Equality compares scalars by value and arrays by ordered element value.

5 Variables and scope

`has` declares a mutable value. In a Joebject body, it declares state. In a behavior or nested brace block, it declares a local value.

```
joebject Meter {
  has jint value = 0

  does add(jint amount) {
    has jint next = value + amount
    value = next
    give value
  }
}
```

Inside a behavior, names resolve in this order: local block values, behavior parameters, owned state, then imported package aliases. A behavior does not silently capture a top-level program variable; pass that value as a parameter. A declaration may not shadow an active local, parameter, or state name.

6 Conditions and loops

6.1 Conditions

```
when score >= 60 {
  show "Pass"
} otherwise {
  show "Try again"
}
```

The condition of `when` must be a `joelean`.

6.2 Counting loops

```
joeach (jint i = 0, i < 5, i++) {
  show i
}
```

The counter exists only inside the loop. `++` increments a `jint` or `jong`; overflow is an error.

6.3 Collection loops

```
has strjin names[] = ["Joe", "Jin", "Eoj"]
joeach name jin names {
  show name
}
```

Elements are visited in increasing index order. The loop variable exists only in the loop body.

7 Fixed Joerays

A fixed Joeray is a typed array with an immutable length. Indexes begin at zero.

```
has jint scores[] = [90, 84, 97]
has jint blank[5]
show scores[0]
scores[1] = 88
show scores.length
```

The only valid index range is `0 <= index < array.length`. Negative indexes and indexes outside this range are errors. Array length is read-only. A `jint` array created with `[5]` contains five zero values; a `strjin` array contains empty strings; a `joelean` array contains `joef`. Assignment overwrites an element but never removes it or changes array length.

8 Joebjects and behaviors

```
joebject Lantern {
  has jint charge = 3
  has strjin label = "Blue lantern"

  does glow() {
    when charge > 0 {
      show label + " is glowing."
      charge = charge - 1
    } otherwise {
      show "The lantern needs charge."
    }
    give charge
  }
}

newjoe Lantern as lantern
show lantern.glow()
```

`newjoe` creates an instance and accepts no arguments in 1.0. Initialize state in its `has` declaration. A behavior may take typed parameters and use `give` to return a result.

```
joebject Scoreboard {
  has jint score = 0
  does add(jint points) {
    score = score + points
    give score
```

```
  }
}
newjoe Scoreboard as board
show board.add(10)
```

Use `self` to call another behavior on the current Joebject.

```
does add_twice(jint amount) {
  self.add(amount)
  give self.add(amount)
}
```

9 Input

`input` is an expression evaluated only while a program runs.

```
has strjin name = input("What is your name?")
has jint age = input("What is your age?")
show "Hello, " + name
```

When runtime reaches `input`, it pauses and issues an input request. The IDE presents the prompt, accepts text after Enter, validates the text against the destination type, assigns the result, then resumes. Invalid numeric or logical input remains active and asks again. Cancellation yields `gunmu` only for a nullable destination; otherwise it produces a required-input cancellation error.

10 Packages

Packages are imported explicitly. The path is a quoted string and an alias is optional.

```
jinport "std/joemath" as math
show math.sqrt(81.0)

jinport "std/joetime"
show joetime.now()
```

When no alias is supplied, the final path segment becomes the package name if it is unique. Missing, incompatible, or unauthorized packages are compile-time errors. Package types are qualified by their alias.

```
jinport "std/joeraylist" as lists
has lists.JoerayList<jint> scores = lists.create<jint>()
```

11 Standard packages

11.1 JoeMath: `std/joemath`

JoeMath is a pure numeric package.

```
jinport "std/joemath" as math
show math.abs(-12)
show math.pow(2.0, 8.0)
show math.clamp(120, 0, 100)
```

Its primary functions are `abs`, `min`, `max`, `floor`, `ceil`, `round`, `sqrt`, `pow`, `clamp`, and seeded `random`.

11.2 JoerayList: `std/joeraylist`

JoerayList is a dynamic, typed list comparable to Java's ArrayList.

```
jinport "std/joeraylist" as lists
has lists.JoerayList<strjin> guests = lists.create<strjin>()
guests.add("Joe")
guests.add("Jin")
show guests.get(0)
guests.set(1, "Eoj")
guests.removeAt(0)
show guests.size()
```

The API includes `add`, `get`, `set`, `removeAt`, `contains`, `clear`, `size`, and `toArray`. All indexes are zero-based and bounds-checked.

11.3 JoeTime: `std/joetime`

JoeTime exposes immutable time values.

```
jinport "std/joetime"
has joetime.TimePoint now = joetime.now()
has joetime.Duration wait = joetime.duration(5000)
has joetime.TimePoint later = joetime.add(now, wait)
show joetime.format(later, "ISO")
```

The main JoeTime functions are `now`, `today`, `parse`, `duration`, `between`, `add`, and `format`. Times use ISO-8601 and UTC by default.

11.4 JoeText: `std/joetext`

JoeText supplies immutable string utilities.

```
jinport "std/joetext" as text
has strjin title = text.trim(" JoeLang ")
show text.toUpper(title)
show text.substring(title, 0, 3)
```

JoeText provides `length`, `charAt`, `substring`, `contains`, `indexOf`, `toUpper`, `toLower`, `trim`, and `split`.

12 Diagnostics

JoeLang reports errors before execution whenever possible. Diagnostics have a stable code, a message, and a source location. A program with an error-level compilation diagnostic does not run. Runtime errors identify their source location and the active Joebject behavior call stack.

13 Quick reference

Form	Meaning
`show expression`	Write output.

Form	Meaning
`has Type name = expression`	Declare a typed mutable value.
`when condition {...}`	Run a conditional block.
`joeach (init, test, update) {...}`	Counting loop.
`joeach item jin collection {...}`	Collection loop.
`Type array[] = [values]`	Fixed typed Joeray.
`joebject Name {...}`	Joebject type declaration.
`does name(Type value) {...}`	Behavior declaration.
`newjoe Name as instance`	Create a Joebject instance.
`give expression`	Return from a behavior.
`input("prompt")`	Request a runtime value.
`jinport "path" as alias`	Activate a package.